

Segunda Parte del Trabajo Final

Teoría de la Computación

Mayo 2026

Sobre la entrega

El uso de Inteligencia Artificial Generativa solo está permitido para la investigación de conceptos y para la generación de ejemplos/casos de prueba. Bajo ningún concepto se aceptarán partes completas generadas con IA. Guardar los links de los chats usados (o capturas), que deberán ser incluidos en el informe final

La fecha límite de entrega será informada oportunamente por Aulas.

En caso de dudas o consultas, deberán utilizar el foro correspondiente.

Se permitirán entregas grupales de hasta tres estudiantes del mismo dictado, siempre que todos los integrantes suban su entrega a Aulas.

La entrega podrá incluir una instancia de defensa presencial.

Objetivo

El objetivo de esta tarea es implementar un intérprete del cálculo lambda puro en Haskell utilizando:

- semántica operacional *small-step*,
- estrategia de evaluación eager (*call-by-value*),
- sustitución captura-evitante,
- sharing explícito.

Ejercicio 1: Representación de términos

Se pide definir una representación algebraica de términos del cálculo lambda puro:

$$M ::= x \mid \lambda x.M \mid MN$$

La representación debe permitir modelar:

- variables,
- abstracciones,
- aplicaciones.

En particular, deberá definirse un tipo del estilo:

```
data Expr
```

Ejercicio 2: Variables libres

Se pide implementar una función:

```
fv :: Expr -> [ String ]
```

que calcule el conjunto de variables libres de un término.

Por ejemplo:

```
fv (\x. x y) = [y]
```

```
fv (\x. \y. x) = []
```

Ejercicio 3: Sustitución captura-evitante

Se pide implementar una función:

```
subst :: String -> Expr -> Expr -> Expr
```

donde:

```
subst x n e
```

representa:

$$e[x := n]$$

La implementación deberá:

- evitar captura de variables,
- realizar renombrado α cuando sea necesario.

Por ejemplo, la sustitución:

$$(\lambda y.x)[x := y]$$

NO puede producir:

$$\lambda y.y$$

Ejercicio 4: Semántica operacional

Se pide implementar una función:

`step :: Expr -> Maybe Expr`

que realice exactamente UN paso de evaluación. Formalice también las reglas en formato de DN que utilice.

La estrategia de evaluación deberá ser:

call-by-value

Es decir:

1. evaluar primero la función,
2. luego el argumento,
3. aplicar β -reducción únicamente cuando el argumento sea un valor.

La implementación deberá respetar las siguientes reglas:

β -reducción

$$(\lambda x.M)V \rightarrow M[x := V]$$

solamente cuando V sea un valor.

Aplicación izquierda

$$M_1 \rightarrow M'_1 \implies M_1 M_2 \rightarrow M'_1 M_2$$

Aplicación derecha

$$M_2 \rightarrow M'_2 \implies V M_2 \rightarrow V M'_2$$

Ejercicio 5: Evaluación completa

Se pide implementar una función:

`eval :: Expr -> Expr`

que evalúe repetidamente utilizando `step` hasta alcanzar una forma normal.

Ejercicio 6: Pretty printer

Se pide implementar una función:

`pretty :: Expr -> String`

que imprima expresiones lambda de manera legible. Instanciar el show de Expr para que use esta función

Ejercicio 7: Sharing

Se pide modificar el intérprete para incorporar sharing explícito.

El objetivo es evitar reevaluaciones innecesarias.

Considere el término:

$$(\lambda x.xx)E$$

Sin sharing, el término E puede evaluarse múltiples veces.

Con sharing, el resultado de evaluar E puede almacenarse y reutilizarse.

La implementación deberá:

- identificar expresiones compartidas,
- reutilizar resultados previamente computados,
- evitar trabajo duplicado.

No existe una única implementación correcta. Investigue opciones, justifique la seleccionada

Ejercicio 8: Numerales de Church

Se pide investigar los numerales de Church e implementar tanto en cálculo lambda puro como embebidos en Haskell:

- representación de 0,
- representación de 1,
- representación de 2,
- sucesor,
- suma,
- multiplicación.

Además, deberán incluir ejemplos de evaluación en Λ embebido.

Ejercicio 9: Comparación experimental

Se pide comparar:

- evaluación SIN sharing,
- evaluación CON sharing.

Para ello deberán utilizarse expresiones donde exista trabajo duplicado.

Puede utilizarse:

```
System.CPUTime
```

o bibliotecas de benchmarking como:

```
criterion
```

Debido a la evaluación lazy de Haskell, la medición deberá forzar la evaluación de resultados.

Por ejemplo:

```
result 'seq' return ()
```

Ejercicio 10: Índices de De Bruijn

Se pide investigar:

- qué son los índices de De Bruijn,
- cuál es su importancia,
- qué problemas intentan resolver,
- ventajas y desventajas respecto a representaciones con nombres.

Además, deberán mostrar informalmente cómo se representarían algunos términos lambda utilizando índices de De Bruijn.